iLink：让 AI 像团队一样工作

AI 研发协作实践分享

汇报人：[填写]

日期：2026年4月

目录

1. 背景：AI 工具已成既成事实
2. 问题：无序使用带来的隐患
3. 方案：iLink 是什么
4. 价值：对公司的具体收益
5. 成本：接入代价几乎为零
6. 结论

一、背景：AI 工具已成既成事实

开发者正在使用 AI，这不是假设

> 当前国内外主流研发工具（Claude Code、Cursor、Copilot、Qoder、Codex）均已深度集成 AI 能力。

事实判断：

- 即便公司没有统一部署，**开发者个人已在使用 AI 辅助编码**
- 国内证券、基金等金融 IT 同行已将 AI 编程纳入研发体系
- AI 工具在**重复性编码**、**文档生成**、**代码审查**上已有显著效率优势

真正的决策不是"用不用 AI"，而是：

> **谁来定义用 AI 的方式，让它在金融级研发环境下可控、可审、可追溯?**

二、问题：无序使用带来的隐患

当前 AI 编程的四个典型风险

#	风险场景	业务影响
1	**AI 超范围修改**：开发者让 AI 改一个接口，AI 顺带修改了 10 个文件	引入隐性变更，测试覆盖不全，上线后出问题
2	**AI 自审自查**：让 AI 检查自己写的代码，AI 很难发现自己的错误	低质量代码进入主干，Code Review 形同虚设
3	**决策不可追溯**：AI 写的代码，对话关掉就没有任何记录	出问题时找不到决策依据，复盘无从下手
4	**团队知识孤岛**：每人用 AI 的方式不同，上下文无法共享	人员变动导致 AI 辅助效果大幅下降

金融行业的特殊压力

> 证券行业对软件的质量压力。代码变更必须可追溯、可解释。

现有 AI 使用模式需要增加人类审核：

- 问："这个字段为什么加密？" → 对话记录已关闭，无法回答
- 问："这段逻辑是谁决策的、基于什么输入？" → 不知道，是 AI 写的
- 要求软件变更要有完整的设计评审记录 → AI 直接写代码跳过了设计环节

这不是 AI 的错，是缺少一套把 AI 融入研发规范的框架。

三、方案：iLink 是什么

一句话定位

> iLink 把 AI 从一个"全能个人"变成一个"四人专业团队"，让 AI 编程过程有角色分工、有文档记录、有人工审核节点。

```
PM（需求分析）→ Designer（技术设计）→ Coder（代码实现）→ QA（代码审查）
      ↓                    ↓                    ↓                    ↓
  业务合同             技术设计文档            代码实现            审查报告
                       ↑ 人工审核节点（Human-Gate）
```

对应传统研发流程：

传统研发	iLink AI 执行	人类角色
产品经理写 PRD	AI PM 输出业务合同	提供需求定义
架构师出设计方案	AI Designer 输出文件清单	**审核设计（最关键）**
开发者写代码	AI Coder 按设计编码	无需介入
同行做 Code Review	AI QA 对照设计审查	最终确认提交

链式审查：每个角色天然审查上游

```
AI PM 读需求 → 输出业务合同
     ↓
AI Designer 读业务合同 → 做技术设计（隐式审查 PM 的理解是否正确）
     ↓
     ⛔ 【人工审核】开发者审核设计 → approve 推进
     ↓
AI Coder 读设计 → 编写代码（只能改设计文档中列出的文件）
     ↓
AI QA 读设计 + 读代码 → 输出审查报告（QA 不知道 Coder 的想法，只比对设计和代码）
     ↓
开发者最终确认 → git commit
```

关键机制：

- Coder **只能修改设计文档明确列出的文件**，硬性约束超范围修改
- QA 是独立 AI 实例，不知道 Coder 的思考过程，相当于真实的同行评审
- **3 次修复不通过自动熔断**，升级人工干预，防止 AI 无限循环
- **STAGING 修订对话**（`ilink-refine`）：当 AI 标记不确定项时，人类可与 AI 逐条确认决策，形成 `[已确认]` 记录

完整审计链：每一步都有文件、有印章

每个需求从提出到上线，产生一条完整的文档链：

```
Jira kcia-1520（登录失败统计接口）
  ├── kcia-1520-需求定义.md      ← 开发者写（人类意图，10分钟）
  ├── kcia-1520-pm.master.md     ← AI PM 输出 + Metadata 印章
  ├── kcia-1520-design.master.md ← AI 设计 + 人类审核记录 + Metadata 印章
  ├── kcia-1520-code.master.md   ← AI 编码记录 + Metadata 印章
  └── kcia-1520-review.master.md ← AI 审查结论 + Metadata 印章
```

Metadata 印章记录：使用的 AI 模型、版本、时间戳、上游文档 SHA1 哈希、变更文件清单

> 问"这个设计决策的依据是什么"，直接翻 `design.master.md`，30 秒内给出答案。

STAGING 修订：不确定的事不硬闯

当 AI 遇到不确定的问题时，会标记 `[待确认]` 并将文档置为 `STAGING`（暂停）。

人类有四种处理方式：

路径	操作
修订对话（`ilink-refine`）	与 AI 逐条确认，AI 记录为 `[已确认 日期: 依据]`
直接编辑	手动修改 STAGING 文档
修改上游重跑	修改需求定义，重新执行对应角色命令
直接推进（`ilink-approve`）	人类主动承担不确定性风险

> 每条 `[已确认]` 记录都是正式的人类决策凭证——出了问题有据可查。

支持主流 AI 工具，不绑定单一平台

AI 工具	如何使用 iLink
Claude Code	`/ilink-pm`、`/ilink-design` 等 slash command
Codex CLI	在对话中输入 `ilink-pm` 等命令
Qoder	`/ilink-pm`、`/ilink-design` 等 slash command

张三用 Claude 做需求 A，李四用 Qoder 做需求 B，共享同一份项目知识库。

这意味着：

- 公司无需统一采购同一 AI 工具
- 个人和团队可根据需要选择工具
- 所有产出进入同一套标准化文档体系

四、价值：对公司的具体收益

核心价值主张：四个维度

价值一：研发效率提升

AI 执行中间环节，人聚焦在高价值决策上

环节	原来耗时	使用 iLink	节省
需求分析整理	2–4 小时	开发者写 10 分钟需求定义，AI 30 秒输出业务合同	~70%
技术设计文档	2–8 小时	AI 1–3 分钟输出，人审核 15 分钟	~60%
编码（中等复杂度）	1–3 天	AI 1–5 分钟，人工审核 20 分钟	~50%
Code Review	30 分钟–2 小时	AI 1–3 分钟，人看审查报告 5 分钟	~70%

> **保守估算：中等复杂度需求的总研发周期缩短 40%–60%。**

价值二：代码质量提升

结构化审查 vs 临时 AI 对话

质量维度	直接 AI 对话	iLink 流水线
需求边界	口头描述，容易歧义	结构化需求定义，有 In/Out Scope
设计评审	无	有设计文档 + 人工 Human-Gate
超范围修改	常见	被文件清单硬约束，不能改列表外的文件
代码审查	AI 自审自查	独立 QA 角色，对照设计文档审查
不确定性处理	AI 默默猜测	`[待确认]` 标记 + `ilink-refine` 人类确认
返工率	高（范围不清、设计偏差）	低（每步有约束和审查）

> **从源头控制 AI 质量，而非事后人工兜底。**

价值三：质量评审能力

金融行业软件质量刚需，降低质量风险

现状问题：

- AI 生成的代码在"聊天框里"，一关窗就消失
- 质量审计时无法提供 AI 决策过程的书面记录
- 设计评审环节被 AI 绕过，质量管控存在漏洞

iLink 的解决方案：

- **每个需求产生完整文档链**，进入 Git 版本控制，永久可查
- **Metadata 印章**记录 AI 版本、时间、上游文档哈希（Upstream_SHA1），不可事后篡改
- **Human-Gate 记录**证明关键节点有人类审核，满足设计评审要求
- `[已确认]` **标记**为每条人类决策留下正式记录，附带日期和依据

> 满足"软件变更应有完整设计评审和变更记录"的质量要求，同时实现了 AI 辅助提效——两者不再矛盾。

价值四：团队知识沉淀

项目知识不随人员变动流失

传统痛点：

- 老员工离职，项目的隐性知识随之消失
- 新员工入职，学习项目架构要靠"老人口口相传"
- AI 对话上下文在个人电脑上，无法团队共享

iLink 的解决方案：

- `project-context.md`：一次生成，全团队共享，AI 每次打开项目自动读取
- **Story 文档链**：每个需求的分析、设计、实现逻辑都有文字记录
- **新成员接手**：读项目知识库 + 翻历史 Story 文档，快速上手

> 知识不再在个人脑袋里，也不再在私人 AI 对话里。**它在 Git 里，属于团队。**

五、成本：接入代价几乎为零

接入成本清单

项目	成本
购置新软件或服务器	**零**
修改现有代码或架构	**零**（iLink 只新增文件，不改动源码）
修改 CI/CD 流水线	**零**
开发者学习新语言/框架	**零**
每个项目的冷启动	**一次，约 5 分钟**（执行 `/ilink-bootstrap`）
开发者日常学习曲线	**记住核心命令，约 30 分钟培训**

最坏情况分析

> 如果试用后效果不佳，退出成本是多少?

退出方案：删除 `iLink/` 目录和 `iLink-doc/` 目录，将 `CLAUDE.md` / `AGENTS.md` 恢复原状。

- 源码不受影响
- 构建配置不受影响
- 线上服务不受影响

结论：这是一个几乎零风险的尝试。

与现有工具的关系

iLink **不是**以下工具的替代品：

现有工具	与 iLink 的关系
Jira	iLink 以 Jira 需求 ID 为 Story ID，两者天然对齐
Git / GitLab	iLink 文档链进入 Git，与代码统一管理
现有 AI 工具（Claude/Copilot 等）	iLink 是这些工具的**使用规范层**，工具不变
代码评审流程（MR/PR）	iLink 的 QA 报告作为 MR 描述的补充，不替代人工 MR
OpenSpec	iLink 是**可叠加的增强层**：OpenSpec 管规格和单次变更，iLink 管 Story 生命周期；文件结构互不冲突，可同时使用
OhMyOpenCode	iLink 是**可叠加的增强层**：OhMyOpenCode 擅长多智能体并行探索，iLink 补充顺序审查链和 Human-Gate；两者解决不同层面的问题

三种方案的定位差异：

方案	核心理念	工作粒度	最适合
OpenSpec	规格驱动开发	单次变更	维护系统规格
OhMyOpenCode	多智能体并行编排	单次任务	快速探索开发

六、结论

核心判断

> AI 辅助编程已是行业现实，问题不是"要不要用"，而是"有没有规范"。

iLink 的价值可以用三句话概括：

1. **效率**：把需求分析、技术设计、编码、代码审查中的重复性工作交给 AI，开发者聚焦在高价值决策上。
2. **质量**：用角色分工约束 AI 的行为边界，链式审查替代 AI 自审自查，Human-Gate 在关键节点保留人工判断。
3. **可追溯**：每个需求产生完整文档链，进入 Git 永久保存，出了问题有据可查，设计决策清晰可回溯。

附录

附录 A：开发者日常使用的命令

核心流水线命令（每个 Story 都用）

命令	用途	频率
`/ilink-init <需求ID>`	创建需求 Story	每个需求一次
`/ilink-pm <需求ID>`	AI 需求分析	每个 Story 一次
`/ilink-design <需求ID>`	AI 技术设计	每个 Story 一次
`ilink-approve <需求ID>`	人类审核推进	审核通过后
`/ilink-coder <需求ID>`	AI 编码实现	设计审核通过后
`/ilink-qa <需求ID>`	AI 代码审查	编码完成后

辅助命令

命令	用途	频率
`/ilink-refine <需求ID>`	修订 STAGING 文档（逐条确认阻塞项）	STAGING 时使用
`ilink-status [需求ID]`	查看流水线状态与下一步建议	随时
`/ilink-bootstrap`	项目冷启动（生成知识库）	每个项目一次

附录 B：iLink 文件结构

```
<项目根目录>/
├── project-context.md          ← 项目知识库（技术栈、模块、构建命令）
├── CLAUDE.md / AGENTS.md       ← 路由文件（引导各 AI 工具读知识库）
├── iLink/                      ← 框架文件（版本统一管理）
│   ├── iLink-root-spec.md
│   ├── iLink-implementation-guide.md
│   └── souls/                  ← 各角色行为规范
└── iLink-doc/                  ← 需求文档链（按 Story ID 归档）
    ├── kcia-1520/              ← 需求 A 的完整文档链
    ├── kcia-1521/              ← 需求 B 的完整文档链
    └── ...
```

所有文件均进入 Git，与代码统一版本管理，审计时随时可查。

附录 C：常见疑问

Q：AI 写的代码质量有保证吗？

A：iLink 的设计前提是 AI 会犯错——这也是为什么有设计审查、有独立 QA、有熔断机制。流水线的目标不是依赖 AI 不犯错，而是通过结构发现和拦截错误。

Q：开发者会不会被 AI 替代？

A：iLink 中人类负责"写需求、审设计、最终把关"，这三件事是 AI 无法替代的。AI 替代的是重复性劳动，人聚焦在判断和决策。

Q：如果某个需求不适合用 iLink 怎么办？

A：iLink 不是强制要求。紧急 bug 修复、配置调整等小变更可以直接操作，不需要走完整流水线。iLink 主要面向需要设计和正式代码审查的需求。

Q：已有项目能接入吗？

A：可以，`/ilink-bootstrap` 命令会分析现有项目结构，自动生成知识库，不修改任何源代码。

Q：AI 标记了 `[待确认]` 怎么办？

A：执行 `/ilink-refine <需求ID>`，与 AI 逐条确认。你的决策会被记录为 `[已确认 日期：依据]`，成为正式的决策凭证。确认完毕后执行 `ilink-approve` 推进流水线。

本材料基于 iLink Root Spec v1.2.00 制作 | 2026年4月